Chapter 8 will cover Spring WebFlux and reactive applications. For now we're only interested in using the `WebTestClient` object to test the API exposed by Catalog Service.

In the previous chapter you saw that Spring Initializr generated an empty `CatalogServiceApplicationTests` class. Let's populate it with integration tests. For this setup, we'll use the `@SpringBootTest` annotation configured to provide a full Spring application context, including a running server that exposes its services through a random port (because it doesn't matter which one).

Listing 3.17 Integration tests for Catalog Service

```
package com.polarbookshop.catalogservice;

import com.polarbookshop.catalogservice.domain.Book;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.web.reactive.server.WebTestClient;
import static org.assertj.core.api.Assertions.assertThat;

@SpringBootTest(                                                    ❶
  webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT
)
class CatalogServiceApplicationTests {

  @Autowired
  private WebTestClient webTestClient;                              ❷

  @Test
  void whenPostRequestThenBookCreated() {
    var expectedBook = new Book("1231231231", "Title", "Author", 9.90);

    webTestClient
      .post()                                                       ❸
      .uri("/books")                                                ❹
      .bodyValue(expectedBook)                                      ❺
      .exchange()                                                   ❻
      .expectStatus().isCreated()                                   ❼
      .expectBody(Book.class).value(actualBook -> {
        assertThat(actualBook).isNotNull();                         ❽
        assertThat(actualBook.isbn())
```